



K. N. Toosi
University of Technology

گزارش پروژه اختیاری ریزپردازنده و زبان اسمبلی

استاد: مهندس دلیر روی فرد

پیاده‌سازی بازی حدس عدد در ATmega64

ابوالفضل شیخ اویسی

شماره دانشجویی: 40120013

مقدمه

هدف این پروژه طراحی و پیاده‌سازی "بازی حدس عدد" با استفاده از میکروکنترلر ATmega64 و زبان اسمبلی AVR بود. این بازی شامل دو بازیکن است که بازیکن اول یک عدد هدف را انتخاب می‌کند و بازیکن دوم تلاش می‌کند این عدد را در ده شانس حدس بزند.

سیستم با استفاده از چهار LED بازخوردهایی برای راهنمایی بازیکن دوم ارائه می‌دهد و محدودیت‌های زمانی دقیقی برای هر مرحله از بازی اعمال می‌کند. این محدودیت‌های زمانی از طریق تایمر صفر تنظیم شده‌اند.

تایمر 0 برای تولید تاخیرهای یک و پنج ثانیه‌ای به کار گرفته شده است که در ادامه به تفصیل درباره جزئیات پیاده‌سازی این تاخیرها صحبت خواهیم کرد.

برای راحتی بازیکن دوم، یک 7-segment که به PortB متصل است، برای نمایش تعداد شانس‌های باقی‌مانده این بازیکن، در نظر گرفته شده است.

نمای کلی کد

1. مقدار دهی اولیه:

- `RESET` : با استفاده از `ORG 0x0000`. مکان شروع کد `INITIAL` را مشخص می‌کنیم.
- `EXT_INT5` : با استفاده از `ORG 0x000C`. مکان شروع کد `EXT_INT5` را مشخص می‌کنیم.
- `TIMO_OVF` : با استفاده از `ORG 0x0020`. مکان شروع کد `TIMO_OVF` را مشخص می‌کنیم.

2. تنظیمات اولیه در `INITIAL` :

- تنظیم `stackpointer` بر روی `0x10FF`
- ست کردن `EIMSK` و `EICRB`
- تعریف ورودی و خروجی بودن پورت‌ها
- خاموش کردن تمام LED ها در ابتدای کار پردازنده
- تنظیم کردن مقدار `prescaler` یا پیش تقسیم‌کننده تایمر 0
- قبول کردن وقفه‌ها
- حلقه انتظار برای شروع بازی

3. وقفه خارجی (`EXT_INT5`) :

- تنظیم عدم پذیرش وقفه خارجی 5 و پذیرش دیگر وقفه‌ها
- دریافت ورودی از بازیکن اول از طریق پورت D
- تاخیر 5 ثانیه‌ای و دریافت ورودی از بازیکن دوم از طریق پورت A
- تصمیم‌گیری بر اساس مقدار ورودی بازیکن دوم با توجه به مقدار هدف و پرش به یکی از سه حالت زیر:

- `CORRECT_GUESS`
- `TARGET_IS_LOWER`
- `TARGET_IS_HIGHER`

4. تایمر 0 (`TIMO_OVF`) :

- استفاده از تایمر برای تولید `wait` های 1 و 5 ثانیه‌ای

1. مقداردهی اولیه آدرس‌ها:

در ابتدای کد با استفاده از **.ORG** آدرس کدهای مدنظر در صورت وقوع وقفه های **Reset**، **INT5**، **timer0_OVF** را تعریف میکنیم.

```
.ORG 0x0000
        JMP INITIAL           // بعد از هر ریست، جامپ کن به آدرس شروع برنامه
.ORG 0x000C
        JMP EXT_INT5         // در صورت وقوع وقفه خارجی 5 ، به آدرس مشخص شده جامپ کن
.ORG 0x0020
        JMP TIM0_OVF         // در صورت وقوع وقفه توسط تایمر 0 ، به آدرس مشخص شده جامپ کن
```

طبق جدول¹ ارائه شده در *datasheet* پردازنده *ATmega*، در صورت بروز وقفه خارجی 5، *ProgramCounter* به آدرس 0x000C در حافظه برنامه می‌رود و از آنجا فرآیند پردازش را ادامه می‌دهد. فضای داده شده برای این کار 2 کلمه یا 4 بایت است که برای پیاده‌سازی دستورات مربوطه فضای کوچکی است. لذا در این آدرس از دستور پرش (JMP) برای جهش به محل دیگری در حافظه برنامه (آدرس INT_5) که فضای قابل استفاده بزرگی دارد استفاده میکنیم.

همین‌طور در صورت وقوع وقفه تایمر یا سرریز تایمر، *ProgramCounter* به آدرس 0x0020 در حافظه برنامه می‌رود. به دلیل مشابه آنچه در توضیح قبل بیان کردیم، از دستور پرش (JMP) برای جهش به آدرس *Timer0_ISR* در حافظه برنامه استفاده میکنیم.

2. تنظیمات اولیه در INITIAL :

در ادامه فلگ INITIAL را تعریف میکنیم و مقدار دهی اولیه را آغاز میکنیم. در این فرآیند مقداردهی *StackPointer*، تنظیمات مربوط به پذیرش وقفه‌ها، تعریف ورودی و خروجی بودن پورت‌ها و همچنین تنظیم *prescaler* تایمر 0 انجام می‌شود که در صفحات بعد به جزئیات آن می‌پردازیم.

¹ Table 23. Reset and Interrupt Vectors ,Page 61, ATmega64 datasheet



INITIAL:

```
LDI R20,low(RAMEND) // مقداردهی هشت بیت کم ارزش استک پوینتر
OUT SPL,R20
LDI R20,high(RAMEND) // مقداردهی هشت بیت پر ارزش استک پوینتر
OUT SPH,R20 // SP = 0x10FF
```

با کمک رجیستر R20، SPL و SPH که به ترتیب هشت بیت کم ارزش و پر ارزش *StackPointer* هستند را مقداردهی کردیم و برابر با آدرس 0x10FF، که آدرس انتهای فضای SRAM است، قرار دادیم.

ادامه INITIAL: ست کردن EIMSK و EICRB

```
LDI R20,0x20 // 0010 0000 ---> Accept Interrupt(5)
OUT EIMSK,R20 // Set External Interrupt Mask Register

LDI R20,0x06 // 0000 1100 ---> ISC5 = 11 --> interrupt(5) trigger by rising edge
OUT EICRB,R20 // Set External Interrupt Control Register B for Interrupt(5)
```

از آنجایی که می‌خواهیم وقفه خارجی 5 پذیرش شود، باید رجیستر *EIMSK* را مقداردهی کنیم و بیت پنجم آن که مخصوص پذیرش وقفه خارجی پنجم است را ست کنیم. به کمک رجیستر R20 و دستور *OUT* که برای مقداردهی آدرس‌های داخل فضای SFR است این کار را انجام می‌دهیم.

در ادامه باید مشخص کنیم که وقفه خارجی 5 تحت چه شرایطی تولید شود. فرض می‌کنیم که با فشردن کلید *Start* که به وقفه خارجی 5 متصل است، سیگنال وقفه از صفر به یک تغییر وضعیت می‌دهد بنابراین حساسیت وقفه خارجی را بر روی لبه بالارونده تنظیم می‌کنیم. طبق جدول² ارائه شده در *datasheet*، برای این کار دو بیت مخصوص آن یعنی بیت‌های *ISC5-0* و *ISC5-1* را که در *EICRB* (*External Interrupt Control Register B*) هستند را ست می‌کنیم. این بیت‌ها به ترتیب بیت‌های *EICRB.2* و *EICRB.3* هستند.

² Table 50. Interrupt Sense Control ,Page 91, ATmega64 datasheet

```
LDI R20,0x00
OUT DDRD,R20      // Define PortD as input --> Player1 Input
OUT DDRA,R20      // Define PortA as input --> Player2 Input

LDI R20,0x0F      // R20 = 0000 1111
OUT DDRC,R20      // Define PortC.0 ... PortC.3 as output | DDRC = 0000 1111
OUT DDRB,R20      // Define PortB.0 ... PortB.3 as output | DDRB = 0000 1111
```

با مقداردهی رجیسترهای DDRA و DDRD با مقدار 0x00، تمام بیت‌های هر دو پورت را به عنوان ورودی تعریف میکنیم. پورت D برای ورودی گرفتن از بازیکن اول و پورت A برای ورودی گرفتن از بازیکن دوم در نظر گرفته شده است.

همچنین از آنجایی که چهار LED به چهار بیت کم‌ارزش پورت C متصل است، چهار بیت کم‌ارزش این پورت را به عنوان خروجی تعیین میکنیم و DDRC را به مقدار 0x0F، مقداردهی میکنیم.

همانطور که در مقدمه گفتیم، از پورت B برای نمایش تعداد شانس‌های باقی مانده بازیکن استفاده میکنیم پس چهار بیت کم‌ارزش آن را به عنوان خروجی تعریف میکنیم.

ادامه INITIAL : خاموش کردن تمام LED ها در ابتدای کار پردازنده

```
LDI R20,0x0F      // Turn off all of the lights
OUT PortC,R20
```

از آنجایی که هر چهار LED به صورت *active-low* هستند، پورت C را به مقدار 0x0F مقداردهی میکنیم تا چهار بیت کم‌ارزش آن که همان چهار بیت متصل به LED ها هستند، *high* شوند تا LED ها در ابتدای کار پردازنده خاموش باشند.

ادامه INITIAL : تنظیم کردن مقدار *prescaler* یا پیش تقسیم‌کننده تایمر 0

```
LDI R20,0x07      // CS0 --> 111
OUT TCCR0,R20      // CLK/1024
```

طبق جدول³ ارائه شده در *datasheet*، مقدار *Clock-Select* بیت‌های تایمر 0 که در رجیستر TCCR0 قرار دارد را برابر 0x07 قرار میدهم تا فرکانس پردازنده بر 1024 تقسیم شود که در ادامه به چرایی این امر می‌پردازیم.

³ Table 56. Clock Select Bit Description, Page 106, ATmega64 datasheet

SEI

// Set Interrupt bit in SREG to accept interrupts

در انتها پس از اتمام تنظیمات مربوط به وقفه‌ها و تعریف شرایط پذیرش وقفه، با استفاده از دستور *SEI*، بیت مخصوص پذیرش *interrupt* یا وقفه، واقع در رجیستر وضعیت *SREG*، را ست می‌کنیم تا از این پس به بعد هر گونه وقفه توسط پردازنده پذیرش و دستورات مربوط به آن انجام شود.

ادامه INITIAL : حلقه انتظار

WAIT_FOR_START:

// loop to wait for EXT_INT5 to start the game

JMP WAIT_FOR_START

برای اینکه پردازنده همواره در فعالیت و منتظر آمدن وقفه برای شروع بازی و فشردن دکمه توسط بازیکن اول بماند، حلقه *WAIT_FOR_START* در نظر گرفته شده است.

3. وقفه خارجی (EXT_INT5)

بخش اصلی منطق بازی حدس عدد، داخل کد این وقفه نوشته شده است. جزئیات پیاده‌سازی هر بخش در ادامه به طور کامل بررسی می‌شود و اکنون صرفاً مختصری از آنچه پیاده‌سازی شده آورده شده است.

توضیح مختصر : ساختار کلی کد این وقفه به این صورت است که در ابتدا و پس از وقوع وقفه، محتویات پورت D که توسط بازیکن اول وارد شده بود، خوانده و در رجیستر R17 نگه‌داری می‌شود. در ادامه و در داخل حلقه *LOOP_10_CHANCE*، تعداد شانس‌های باقی‌مانده بازیکن دوم که در رجیستر R18 نگه‌داری شده، بررسی می‌شود و طبق صورت مسئله پروژه، حداکثر 10 بار به بازیکن دوم فرصت داده می‌شود تا حدس خود را وارد کند.

داخل حلقه *LOOP_10_CHANCE* ابتدا 5 ثانیه به بازیکن دوم فرصت داده می‌شود تا حدس خود را وارد کند که در این فاصله چراغ‌های سفید روشن می‌مانند. پس از این مقداری که بازیکن دوم در پورت A وارد کرده، خوانده می‌شود و با مقدار هدف (مقداری که بازیکن اول وارد کرده) مقایسه می‌شود.

در صورت برابری این دو مقدار به *CORRECT_GUESS* جهش می‌کنیم و به مدت 5 ثانیه چراغ سبز را روشن می‌کنیم و به حلقه انتظار برمی‌گردیم و منتظر فشردن مجدد دکمه *start* و وقفه خارجی 5 می‌شویم.



اما در صورتی که مقدار هدف از مقداری که وارد شده بیشتر باشد، به *TARGET_IS_HIGHER* جهش میکنیم و به مدت 1 ثانیه چراغ قرمز را روشن نگه می‌داریم.

همچنین در صورت کمتر بودن مقدار هدف از مقدار وارد شده، به *TARGET_IS_LOWER* جهش کرده و چراغ زرد را به مدت 1 ثانیه روشن نگه می‌داریم.

پس از سپری شدن 1 ثانیه و راهنمایی بازیکن دوم در هر دو حالت بالا، به *LOOP_10_CHANCE* بر می‌گردیم و دوباره به بازیکن دوم تا زمانی که شانس‌هایش تمام نشده، فرصت می‌دهیم تا حدس بعدی خود را وارد کند.

در صورتی که حدس‌های بازیکن تمام شود و مقدار R18 به صفر برسد، به *LOST_GAME* جهش می‌کنیم و به مدت 5 ثانیه تمام چراغ‌ها را روشن نگه می‌داریم که به منزله باختن بازیکن دوم است و دوباره به حلقه تکرار برمی‌گردیم و منتظر شروع مجدد بازی و آمدن وقفه خارجی 5 می‌مانیم.

جزئیات پیاده‌سازی وقفه خارجی (EXT_INT5) :

```
----- //Interrupt 5 code started-----  
EXT_INT5:  
    LDI R20,0x00          // 0000 0000 --> deny Interrupt(5)  
    OUT EIMSK,R20         // Set External Interrupt Mask Register  
    SEI  
    IN R17,PIND           // Store PlayerA input value into R17 ( target_value )  
    LDI R18,10            // Load maximum PlayerB chances into R18
```

در ابتدا مقدار رجیستر *EIMSK* را به 0x00 تغییر می‌دهیم تا دیگر وقفه خارجی پنج پذیرش نشود. در ادامه با استفاده از دستور *SEI* بیت پذیرش وقفه را ست می‌کنیم تا بتوان از وقفه‌های تایمر استفاده کرد اما وقفه‌های *EXT_INT5* پذیرش نخواهند شد.

بلافاصله پس از وقوع وقفه، مقدار وارد شده توسط بازیکن اول، در رجیستر R17 ذخیره می‌شود و تا انتهای بازی مقدار این رجیستر بدون تغییر باقی می‌ماند.

پس از آن تعداد شانس‌های بازیکن دوم برای حدس عدد وارد شده که طبق صورت پروژه 10 شانس است، در رجیستر R18 نگه‌داری می‌شود. در ادامه خواهیم دید که در *LOOP_10_CHANCE* مقدار این رجیستر پس از هر تلاش بازیکن اول یکی کم می‌شود و در صورتی که شانس‌های بازیکن دوم تمام شود، بازی به اتمام می‌رسد.

LOOP_10_CHANCE:

```

OUT PORTB, R18          // Refresh PlayerB chances

CPI R18,0

BREQ LOST_GAME

DEC R18

LDI R20,0x0E            // Turn on white light --> portC.0 = 0 | PortC = 0000 1110

OUT PortC,R20

*****

5 seconds wait time

*****

IN R16,PINA             // Store PlayerB input value into R16

CP R17,R16

BREQ CORRECT_GUESS      // If (input_by_playerB == target_value) --> jmp CORRECT

BRLO TARGET_IS_LOWER    // If (input_by_playerB > target_value) --> jmp TARGET_IS_LOWER

TARGET_IS_HIGHER:

***

** TARGET_IS_HIGHER code **

***

JMP LOOP_10_CHANCE

```

در حلقه *LOOP_10_CHANCE* ، در ابتدای کار مقدار رجیستر R18 با صفر مقایسه می‌شود تا در صورت اتمام شانس‌های بازیکن دوم، بازی خاتمه یابد و به فلگ *LOST_GAME* جهش کند. اما در صورتی که بازیکن دوم هنوز شانس داشته باشد، یکی از مقدار شانس‌های بازیکن دوم کم می‌شود و به او فرصت داده می‌شود.

در ادامه مقدار 0x0E به پورت C داده می‌شود تا فقط چراغ سفید روشن شود که به معنی انتظار برای وارد کردن حدس بعدی بازیکن دوم است. پس از 5 ثانیه انتظار (توضیحات پیاده‌سازی 5 ثانیه در صفحه 14 و 15 به طور کامل) ، مقداری که بازیکن دوم در پورت A وارد کرده خوانده می‌شود و در رجیستر R16 ریخته می‌شود. سپس مقدار R17 که مقدار هدف است با مقدار R16 که حدس بازیکن دوم است با هم مقایسه می‌شوند تا در مورد ادامه روند بازی تصمیم‌گیری شود.

در صورتی که این دو مقدار برابر باشد، شرط *BREQ* برقرار می‌شود و به *CORRECT_GUESS* جهش میکنیم که در ادامه پیاده‌سازی شده است. اما اگر مقدار هدف از حدس بازیکن دوم کمتر باشد، به *TARGET_IS_LOWER* جهش میکنیم که جلوتر به جزئیات آن می‌پردازیم.

و در آخر اگر هیچ یک از این دو حالت نبود، بدیهی است که مقدار هدف از حدس بازیکن بزرگتر است و وارد کد *TARGET_IS_HIGHER* می‌شویم. در انتهای دو حالت نامساوی، با *JMP* به *LOOP_10_CHANCE* باز می‌گردیم.



TARGET_IS_HIGHER:

```
LDI R20,0x07          // Turn on Red light --> portC.3 = 0 | PortC = 0000 0111
OUT PortC,R20
// *****
// 1 second wait time
// *****
LDI R20,0x0F          // Turn off Red light after 1 second --> portC.3 = 0 | PortC= 0000 1111
OUT PortC,R20
JMP LOOP_10_CHANCE
```

زمانی که برنامه به فلگ *TARGET_IS_HIGHER* جهش میکند، به این معنی است که مقدار هدف، از حدسی که بازیکن دوم وارد کرده بزرگتر است. در نتیجه طبق صورت مسئله باید با یک ثانیه روشن نگه داشتن چراغ قرمز، بازیکن دوم را راهنمایی کنیم.

برای این کار به کمک رجیستر R20، مقدار 0x07 را در پورت C میریزیم تا تنها LED قرمز رنگ روشن شود. سپس با استفاده از تایمر 0 و کدی که پیاده‌سازی کردیم و به جزئیات آن خواهیم پرداخت، 1 ثانیه صبر میکنیم و پس از 1 ثانیه، با ریختن مقدار 0x0F در پورت C، LED قرمز را خاموش میکنیم. (باقی LED ها نیز خاموش خواهند بود) در انتها پس از اینکه بازیکن دوم راهنمایی لازم را دریافت کرد، به *LOOP_10_CHANCE* باز می‌گردیم تا بازی ادامه یابد.

جزئیات پیاده‌سازی 1 ثانیه مکث و wait در صفحه 14 و 15 به طور کامل توضیح داده شده است.

**** نکته مهم **** هنگام استفاده از دستورات شرطی *Branch*، باید این نکته را در نظر داشته باشیم که این دستورات تنها میتوانند به 63 دستور قبل‌تر یا 64 دستور بعد‌تر از خود جهش کنند و بیشتر از این مقدار را نمیتوانند جهش کنند. دلیل این اتفاق این است که کد مربوط به آدرس جهش در دستورات شرطی، حداکثر 6 بیتی است.

به همین علت مجبور شدیم به ناچار کد مربوط به *LOST_GAME* را بعد از *TARGET_IS_HIGHER* قرار دهیم تا به *BREQ LOST_GAME* که در *LOOP_10_CHANCE* به کار بردیم، نزدیک باشد.



LOST_GAME:

```
LDI R20,0x00          // Turn on all the lights --> PortC = 0000 0000
OUT PortC,R20
// *****
// 5 seconds wait time
// *****
LDI R20,0x0F          // Turn off lights after 5 seconds --> PortC = 0000 1111
OUT PortC,R20
```

GAME_FINISHED:

```
LDI R20,0x20          // 0010 0000 --> Accept Interrupt(5)
OUT EIMSK,R20         // Set External Interrupt Mask Register
RETI
```

هنگامی که بازیکن دوم از تمام شانس‌هایش استفاده کرده باشد، مقدار R18 صفر می‌شود به معنی باختن بازیکن دوم در بازی حدس عدد است و به فلگ *LOST_GAME* جهش می‌کنیم. طبق صورت مسئله پس از ناموفق بودن بازیکن دوم در حدس عدد هدف بعد از 10 تلاش، بایستی تمام *LED* ها به مدت 5 ثانیه روشن بمانند و سپس بازی به اتمام برسد و منتظر وقفه خارجی 5 یا همان کلید *start* بمانیم تا بازی مجدداً توسط بازیکن اول شروع شود.

برای همین ابتدا مقدار 0x00 را در پورت *C* می‌ریزیم که به علت *active-low* بودن *LED* ها، به منزله روشن کردن تمام *LED* ها است. سپس به مدت 5 ثانیه با استفاده از کد پیاده‌سازی شده با کمک تایمر 0، صبر می‌کنیم و پس از 5 ثانیه، با ریختن مقدار 0x0F در پورت *C*، تمام *LED* ها را خاموش می‌کنیم.

قبل از اینکه با *RETI* به حلقه انتظار در بخش *INITIAL* بازگردیم، مقدار *EIMSK* را به 0x20 تغییر می‌دهیم تا از این به بعد وقفه *EXT_INT5* که در ابتدای کد *EXT_INT5* غیرفعالش کرده بودیم، پذیرش شود.

از آنجایی که از *RETI* استفاده کردیم پس بعد از اتمام بازی، با ست شدن بیت مخصوص *interrupt* در رجیستر وضعیت یا همان *status-register(SREG)*، وقفه‌های جدید می‌توانند پذیرفته شوند.

جزئیات پیاده‌سازی 1 ثانیه مکث و *wait* در صفحه 14 و 15 به طور کامل توضیح داده شده است.

TARGET_IS_LOWER:

```
LDI R20,0x0B          // Turn on Yellow light --> portC.2 = 0 | PortC = 0000 1011
OUT PortC,R20

// *****

// 1 second wait time

// *****

LDI R20,0x0F          // Turn off Yellow light after 1 second --> PortC = 0000 1111
OUT PortC,R20

JMP LOOP_10_CHANCE
```

زمانی که برنامه به فلگ *TARGET_IS_LOWER* جهش میکند، به این معنی است که مقدار هدف، از حدسی که بازیکن دوم وارد کرده کوچکتر است. در نتیجه طبق صورت مسئله باید با یک ثانیه روشن نگه داشتن چراغ زرد، بازیکن دوم را راهنمایی کنیم تا در شانس‌های بعدی، مقادیر کوچک‌تری وارد کند تا بتواند برنده شود.

برای این کار به کمک رجیستر R20، مقدار 0x0B را در پورت C میریزیم تا تنها LED زرد رنگ روشن شود. سپس با استفاده از تایمر 0 و کدی که پیاده‌سازی کردیم و به جزئیات آن خواهیم پرداخت، 1 ثانیه صبر میکنیم و پس از 1 ثانیه، با ریختن مقدار 0x0F در پورت C، LED زرد را خاموش میکنیم. (باقی LED ها نیز خاموش خواهند بود)

در انتها پس از اینکه بازیکن دوم راهنمایی لازم را دریافت کرد، به *LOOP_10_CHANCE* باز می‌گردیم تا بازی ادامه یابد.

جزئیات پیاده‌سازی 1 ثانیه مکث و *wait* در صفحه 14 و 15 به طول کامل توضیح داده شده است.

CORRECT_GUESS:

```
LDI R20,0x0D          // Turn on Green light --> portC.1 = 0 | PortC = 0000 1101
OUT PortC,R20
// *****
// 5 seconds wait time
// *****
LDI R20,0x0F          // Turn off Green light after 5 seconds --> PortC = 0000 1111
OUT PortC,R20

LDI R20,0x20          // 0010 0000 --> Accept Interrupt(5)
OUT EIMSK,R20         // Set External Interrupt Mask Register
RETI
```

زمانی که بازیکن دوم بتواند در یکی از 10 شانس که دارد، مقدار هدف را به درستی حدس بزند، وارد فلگ *CORRECT_GUESS* می‌شویم. طبق صورت مسئله در صورت برنده شدن بازیکن دوم و حدس درست، باید LED سبز رنگ به مدت 5 ثانیه روشن شود و سپس بازی به اتمام برسد و دوباره منتظر شروع بازی با فشردن کلید *start* یا همان وقفه خارجی باشد.

به همین علت در ابتدا مقدار 0x0D در پورت C ریخته می‌شود تا LED سبز رنگ که به بیت PORTC.1 متصل است و به صورت *active-low* کار میکند، روشن شود. سپس با استفاده از کد پیاده‌سازی شده به کمک تایمر 0، به مدت 5 ثانیه صبر میکنیم و پس از 5 ثانیه، همه LED ها را خاموش میکنیم.

قبل از اینکه با *RETI* به حلقه انتظار در بخش *INITIAL* بازگردیم، مقدار *EIMSK* را به 0x20 تغییر می‌دهیم تا از این به بعد وقفه *EXT_INT5* که در ابتدای کد *EXT_INT5* غیرفعالش کرده بودیم، پذیرش شود.

توجه شود که چون با *RETI* برمی‌گردیم، مجدداً بیت پذیرش وقفه ست می‌شود و وقفه‌ها میتوانند پذیرش شوند.

4. تایمر 0 (TIM0_OVF)

برای پیاده‌سازی انتظارهای 1 و 5 ثانیه‌ای از تایمر 0 استفاده می‌شود. در پیاده‌سازی پروژه فرکانس کاری پردازنده ATmega64، برابر با 1.024 MHz در نظر گرفته شده که در ادامه می‌بایست با توجه به این فرکانس، برای تولید انتظارهای 1 و 5 ثانیه‌ای برنامه‌ریزی کنیم.

در بخش INITIAL، مقدار prescaler را مطابق کد زیر، برابر با 1024 قرار دادیم تا بتوانیم به دقت ثانیه نزدیک شویم.

```
LDI R20,0x07          // CS0 --> 111
OUT TCCR0,R20          // CLK/1024
```

طبق جدول⁴ ارائه شده در datasheet، مقدار Clock-Select بیت‌های تایمر 0 که در رجیستر TCCR0 قرار دارد را برابر 0x07 قرار می‌دهیم تا فرکانس پردازنده بر 1024 تقسیم شود.

طبق فرمول‌هایی که در صفحه 12 فصل هفتم جزوه ارائه شد، به محاسبه مقادیر می‌پردازیم. برای محاسبه فرکانس تایمر با توجه به مقدار prescaler، باید فرکانس کاری CPU را بر 1024 تقسیم کنیم تا فرکانس تایمر بدست آید.

$$F_{\text{Timer}} = F_{\text{CPU}} / 1024$$

$$F_{\text{Timer}} = 1.024 \text{ MHz} / 1024 = 1 \text{ KHz}$$

پس فرکانس تایمر برابر با 1 KHz خواهد بود. برای محاسبه T_{Timer} داریم:

$$T_{\text{Timer}} = 1 / F_{\text{Timer}}$$

$$T_{\text{Timer}} = 1 / 1 \text{ KHz} = 1 \text{ ms}$$

بدین ترتیب مقدار شمارنده هر 1 ms یک واحد افزایش می‌یابد. برای محاسبه تعداد شمارش مورد نیاز توسط تایمر داریم:

زمان افزایش یک واحد / زمان دلخواه = تعداد شمارش

زمان دلخواه ما 1 ثانیه و زمان افزایش یک واحدی مقدار شمارنده نیز، 1 ms است. در نتیجه تعداد شمارش بایستی برابر با 1000 باشد یعنی شمارنده 1000 بار بشمارد و سپس سرریز کند و وقفه را ارسال کند. اما تایمر حداکثر تا 255 می‌تواند شمارش کند.

⁴ Table 56. Clock Select Bit Description, Page 106, ATmega64 datasheet



برای حل این مشکل، ابتکار عمل به خرج می‌دهیم و زمان دلخواه‌مان را از 1s به 250ms کاهش می‌دهیم. در این صورت تایمر بایستی تا 250 شمارش کند و سپس وقفه ارسال کند:

زمان افزایش یک واحد / زمان دلخواه = تعداد شمارش

زمان دلخواه = 250 ms

زمان افزایش یک واحد = 1 ms

$$\text{Number_of_Counts} = 250 \text{ ms} / 1 \text{ ms} = 250$$

برای اینکه مقدار ابتدایی عدد شمارنده را بدست آوریم داریم :

$$\text{Timer_first_load} = 256 - \text{Number_of_Counts} = 256 - 250 = 6$$

در نتیجه باید در رجیستر *TCNT0* مقدار 6 را قرار دهیم که تایمر از 6 شروع کند و پس از سپری شدن 250 تا شمارش، وقفه را ارسال کند.

از آنجایی که تایمر هر 250ms یک وقفه ارسال میکند و ما به دنبال صبر کردن برای 1 و 5 ثانیه هستیم، نیاز هست تا ابتکار به خرج دهیم. به این صورت که بعد از هر 4 باری که وقفه تایمر ارسال شد، یک ثانیه را شمارش کنیم و در داخل رجیستری زمان سپری شده را نگهداری کنیم.

در حقیقت می‌توان گفت که با این ابتکار، هر 4 وقفه تایمر، برابر با 1 ثانیه خواهد بود.

توضیحات بیشتر را در صفحه بعد روی کد وقفه تایمر (*TIMERO_OVF*) ارائه خواهیم کرد.

TIM0_OVF:

```
LDI R20,6           // 256 - 250 = 6
OUT TCNT0, R20      // Count from 6

CPI R19,4
BRNE NOT_A_SEC      // if (R19 < 4) --> JMP NOT_A_SEC
                    // else if (R19 == 4) --> CLR R19 and count one second

CLR R19
INC R21              // Count 1 second
JMP END_TIMER

NOT_A_SEC:
INC R19

END_TIMER:
RETI
```

پس از گذشت 250ms بعد از فعال کردن تایمر 0 و زمانی که این تایمر سرریز می‌کند، *ProgramCounter* به آدرسی که فلگ *TIM0_OVF* به آن اشاره می‌کند می‌آید.

در ابتدای کار باید دوباره رجیستر *TCNT0* را به مقدار 6 لود کنیم که برای وقفه بعدی نیز از 6 شروع به شمارش کند و 250 تا بشمارد و بعد وقفه ارسال کند.

رجیستر R19 را رزرو میکنیم و به صورت اختصاصی برای شمارش تعداد وقفه‌هایی که توسط تایمر تولید شده استفاده میکنیم. لازم به ذکر است که قبل از شروع شمارش و فعال کردن تایمر، رجیستر R19، CLR میشود.

چون می‌دانیم که فاصله زمانی ای که 4 وقفه توسط تایمر 0 تولید می‌شود برابر 1 ثانیه است پس هر سری که وقفه تولید شد، مقدار R19 را با 4 مقایسه میکنیم.

در صورتی که مقدار R19 برابر با 4 بود، یعنی یک ثانیه سپری شده و مقدار رجیستر R21 که برای شمارش تعداد ثانیه‌های سپری شده رزرو کردیم را یک واحد افزایش میدهیم. رجیستر R21 هم پیش از فعال کردن تایمر و در بدنه کد اصلی CLR می‌شود که در ادامه نشان خواهیم داد.

اما در صورتی که مقدار R19 از 4 کمتر باشد، یعنی هنوز یک ثانیه سپری نشده و به *NOT_A_SEC* پرش میکنیم که مقدار R19 که همان شمارش تعداد وقفه‌های تایمر است را یک واحد افزایش می‌دهیم.

در آخر هم با استفاده از *RETI* به نقطه‌ای که وقفه در آنجا رخ داد برمیگردیم.

برای درک بهتر یک نمونه از انتظارهای پنج ثانیه‌ای که در *LOOP_10_CHANCE* استفاده شده را توضیح می‌دهم.

LOOP_10_CHANCE:

```
...                // بخش ابتدایی کد
// 5 seconds wait time
CLR R19
CLR R21
LDI R20,6          // 256 - 250 = 6
OUT TCNT0, R20     // Count from 6
LDI R20,0x01
OUT TIMSK, R20     // Enable to Timer Interrupt
```

WAIT_5_loop1:

```
CPI R21,5
BRNE WAIT_5_loop1
LDI R20,0x00
OUT TIMSK, R20    // Disable to Timer Interrupt
CLR R19
CLR R21           // reset seconds
// Finish 5 seconds wait time
...              // ادامه کد
```

همان‌طور که در بخش *LOOP_10_CHANCE* توضیح دادیم، بعد از روشن کردن *LED* سفید، باید 5 ثانیه به بازیکن دوم فرصت بدهیم تا حدسش را وارد کند و پس از گذشت این 5 ثانیه، *LED* سفید را خاموش کنیم و ورودی بازیکن دوم را از پورت A بخوانیم.

در کد بالا جزئیات پیاده سازی این 5 ثانیه انتظار که در قبل به آن پرداخته نشد و صرفاً به آن اشاره شد را مشاهده می‌کنید. در ابتدای کار رجیسترهای *R19* و *R21* که قرار است به ترتیب به عنوان شمارنده تعداد وقفه‌های تایمر 0 و شمارنده ثانیه‌های سپری شده استفاده شود را *clear* میکنیم. در ادامه باید مقدار 6 را در *TCNT0* لود کنیم که شمارش را از 6 آغاز کند و 250 تا بشمارد تا هر 250 ms وقفه تولید شود.

در خط بعد مقدار رجیستر *TIMSK* را برابر 0x01 قرار میدهم تا وقفه‌های تولید توسط تایمر 0 توسط پردازنده پذیرش شود. بعد از اعمال تنظیمات اولیه در حلقه *WAIT_5_loop1*، منتظر میمانیم تا 5 ثانیه سپری شود. نحوه کار به این صورت است که در ابتدای حلقه رجیستر *R21* که حاوی ثانیه‌های سپری شده بود، با 5 مقایسه میشود و در صورتی که هنوز به 5 نرسیده باشد، دوباره به *WAIT_5_loop1* جامپ می‌کند تا بالاخره 5 ثانیه سپری شود و از حلقه خارج شود.

لازم به ذکر است که مقدار *R21* داخل برنامه وقفه تایمر 0 بعد از هر 4 وقفه تایمر یک واحد افزایش پیدا میکند. بعد از اینکه پنج ثانیه سپری شد، مقدار *TIMSK* را برابر 0x00 قرار میدهم تا دیگر وقفه تایمر پذیرش نشود و به ادامه کد می‌رسیم. در نتیجه توانستیم بین دو عملیات داخل کد، 5 ثانیه تاخیر ایجاد کنیم.



دقیقا مشابه آنچه که در *LOOP_10_CHANCE* برای پیاده‌سازی *wait* به مقدار 5 ثانیه انجام دادیم را برای پیاده‌سازی *wait* به مقدار 1 ثانیه هم انجام می‌دهیم با این تفاوت که این بار در حلقه *WAIT_1_loop#* باید مقدار *R21* را به جای 5، با 1 مقایسه کنیم تا پس از 1 ثانیه از انتظار خارج شویم و به ادامه کد برسیم.

TARGET_IS_HIGHER:

```
LDI R20,0x07 //Turn on Red light --> portC.3 = 0 | PortC = 0000 0111
```

```
OUT PortC,R20
```

```
***** //
```

```
1 //second wait time
```

```
CLR R19
```

```
CLR R21
```

```
LDI R20,6 // 256 – 250 = 6
```

```
OUT TCNT0, R20 // Count from 6
```

```
LDI R20,0x01
```

```
OUT TIMSK, R20 // Enable to Timer Interrupt
```

WAIT_1_loop1:

```
CPI R21,1
```

```
BRNE WAIT_1_loop1
```

```
LDI R20,0x00
```

```
OUT TIMSK, R20 // Disable to Timer Interrupt
```

```
CLR R19
```

```
CLR R21
```

```
***** //
```

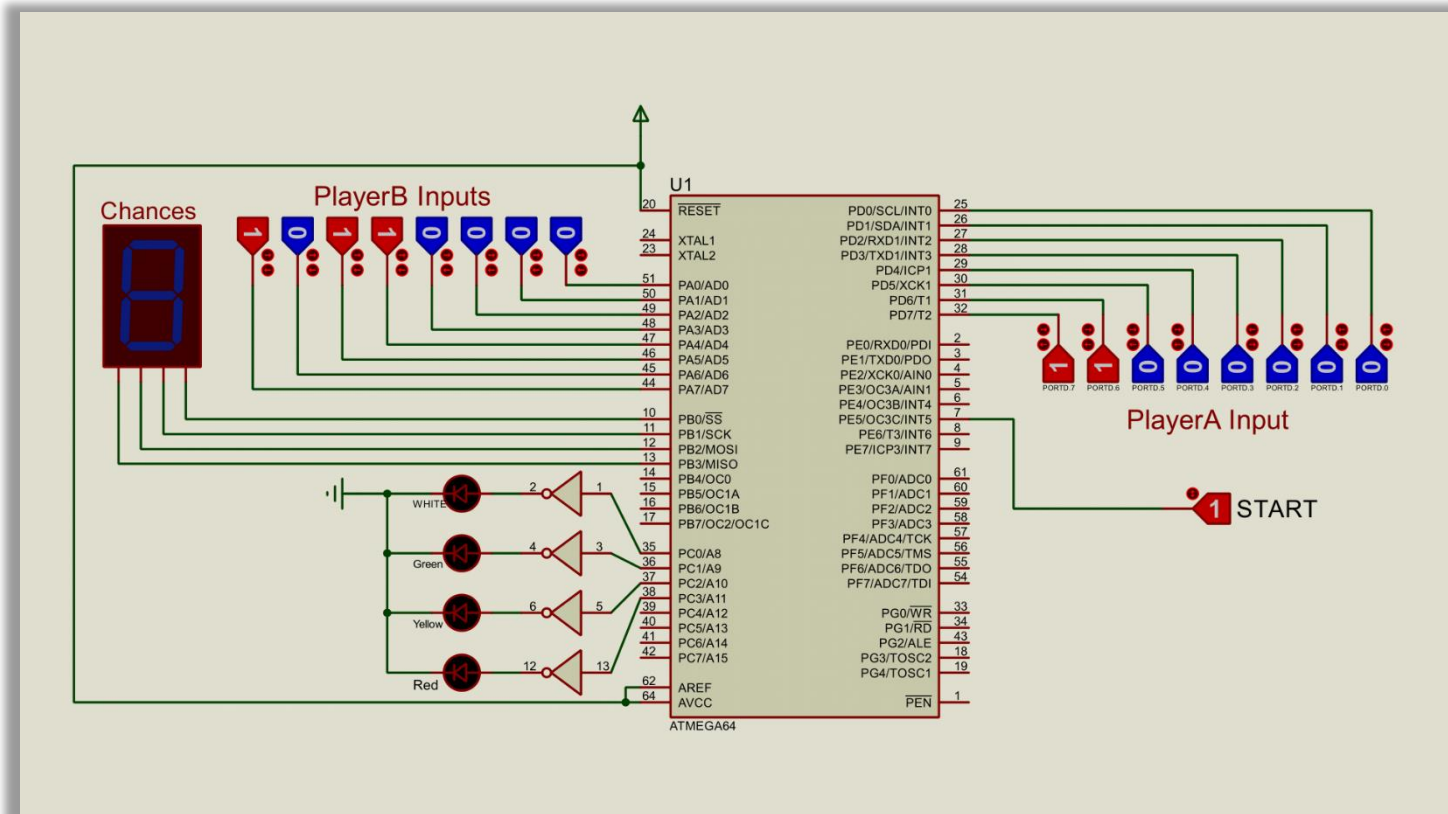
```
LDI R20,0x0F //Turn off Red light after 1 second --> portC.3 = 0 | PortC = 0000 1111
```

```
OUT PortC,R20
```

```
JMP LOOP_10_CHANCE
```

همانطور که گفته شد، در مثال *TARGET_IS_HIGHER*، پورت C مقدار دهی می‌شود تا LED قرمز روشن شود و در *WAIT_1_loop1* هی می‌چرخیم و منتظر می‌مانیم تا یک ثانیه سپری شود و پس از گذشت یک ثانیه، از حلقه خارج می‌شویم و پورت C را مقداردهی مجدد می‌کنیم تا LED قرمز خاموش شود و به *LOOP_10_CHANCE* پرش می‌کنیم.

تست کد در نرم افزار Proteus



همانطور که در تصویر مشخص است، از یک پردازنده *Atmega64* استفاده میکنیم و *source-code* مربوط به پروژه را که پیشتر مفصل راجع به آن حرف زدیم، به عنوان حافظه برنامه به آن می‌دهیم.

در بالا سمت راست، تعداد هشت *LogicState* برای گرفتن ورودی از بازیکن اول در نظر گرفته شده که طبق صورت مسئله، به پورت *D* پردازنده متصل است و عدد ورودی بازیکن اول را به پردازنده می‌دهد.

هم چنین یک *LogicToggle* پایین تر از آن مشاهده می شود که به عنوان دکمه شروع در نظر گرفته شده که به *interrupt* یا وقفه شماره 5 متصل است. این *LogicToggle* در ابتدا مقدار 1 را دارد و وقتی فشرده می شود، از 1 به 0 تغییر می کند که منجر به فعال شده وقفه خارجی 5 در لبه پایین رونده می شود و بازی شروع می شود.

اکنون LED سفید رنگ که در سمت چپ پردازنده و در پایین مشخص است، به مدت پنج ثانیه روشن می ماند تا بازیکن دوم، حدس خود را وارد کند. در این مدتی که LED سفید رنگ روشن است، بازیکن دوم فرصت دارد حدس خود را به وسیله هشت عدد LogicState ای که در بالا سمت چپ پردازنده مشاهده می شود و به پورت A متصل است، وارد کند.

پس از گذشت 5 ثانیه مقدار PinA توسط پردازنده خوانده و با مقدار هدف مقایسه می‌شود و با توجه به حدس بازیکن دوم، اگر اشتباه باشد، یکی از شانس‌های بازیکن دوم کاسته میشود و LED زرد یا قرمز، به مدت 1 ثانیه روشن می‌ماند تا بازیکن دوم را راهنمایی کند و سپس دوباره 5 ثانیه LED سفید روشن می‌شود تا بازیکن دوم حدس بعدی خود را وارد کند. اما اگر حدس بازیکن دوم درست باشد، چراغ سبز به مدت 5 ثانیه روشن می‌ماند و بازی تمام می‌شود.



همچنین در صورتی که بازیکن دوم بعد از ده شانس نتواند حدس درست بزند، بازی به اتمام می‌رسد و هر چهار LED سفید، سبز، زرد و قرمز به مدت 5 ثانیه روشن می‌مانند و بازی به اتمام می‌رسد.

لازم به ذکر است که پس از اتمام بازی، بازیکن اول میتواند دوباره یک عدد در پورت D وارد و با فشردن دکمه *Start*، بازی را شروع کند.

ویدیوی تست پروژه در نرم افزار Proteus در فایل زیر ارسال می‌باشد.